Version corrigée du 2025-02-11

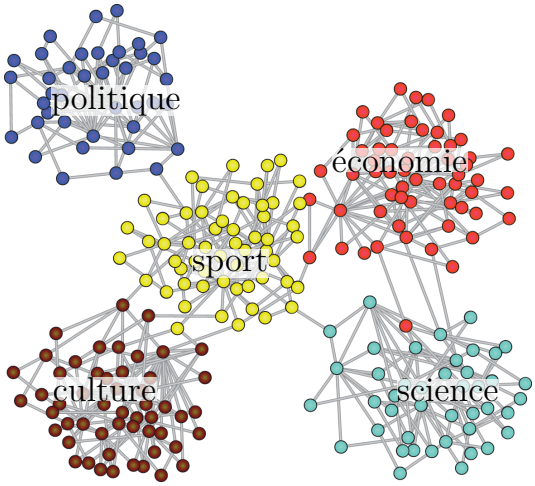
TD 1

Intelligence artificielle

Exercices

Exercice 1 : Apprentissage supervisé et classement d'articles de presse . . . . . [⋆]

Des informaticiens travaillant pour un média connu souhaitent élaborer un algorithme de classement des articles de presse pour le site internet du magazine. Les articles de presse se répartissent en 5 sujets : sport, politique, science, culture, économie. Pour que l'algorithme soit capable de classer un nouvel article de presse dans l'une de ces 5 classes, on procède par apprentissage supervisé et on recourt à l'algorithme des k plus proches voisins.



Classification — On dispose d'un ensemble d'apprentissage `articles` contenant les $N = 280$ articles de presse déjà publiés. On souhaite créer une classification des articles de presse selon leur sujet. Pour cela, on modélise un article de presse par un point du plan $y = (e, n)$ où $0 \leq e \leq 1$ représente la quantité d'équations dans le texte et $0 \leq n \leq 1$ est la quantité de noms propres.

1) Rappeler le principe de l'apprentissage supervisé et d'un algorithme de classement.
2) On donne dans le lien ci-dessous une base de données contenant l'ensemble d'apprentissage `articles` (liste de points) et la classification `sujets` (de type dictionnaire). Que vaut `sujets[(0.33, 0.31)]` ?

https://cahier-de-prepa.fr/pc-theo/download?id=682

3) Définir en `python` une fonction distance `d` qui prend comme arguments deux points `y1` et `y2` du plan et qui renvoie la distance euclidienne entre ces deux points.

Solution

```python
def tri(E, x, d):
    """
    Entrée : ensemble d'apprentissage E (list), point x (tuple),
             distance d (function)
    Action : tri l'ensemble E selon la distance des points à x
    Sortie : ensemble E trié (list)
    """
    return sorted(E, key = lambda y:d(x,y))
```

Phase d'apprentissage : algorithme des k plus proches voisins — On souhaite implémenter puis appliquer l'algorithme des k plus proches voisins pour classer le nouvel article $x = (0.5, 0.6)$. On note de façon générique E un ensemble d'apprentissage, p une classification, k entier le nombre de voisins considérés et d la distance.

4) Rappeler le principe de l'algorithme des k plus proches voisins. Que représentent les fonctions $\text{tri}(E, x, d)$, $\text{voisins}(E, x, d, k)$ et $\text{classe}(E, x, d, k, p)$?

5) On implémente la fonction $\text{tri}(E, x, d)$ de la façon suivante. Vérifier le fonctionnement de cette fonction sur des exemples simples, par exemple pour `E=[(0.1,0.1),(0.9,0.9),(0.5,0.5)]` et `x=(0.4,0.6)`.

```python
def tri(E, x, d):
    """
    Entrée : ensemble d'apprentissage E (list), point x (tuple),
             distance d (function)
    Action : tri l'ensemble E selon la distance des points à x
    Sortie : ensemble E trié (list)
    """
    return sorted(E, key = lambda y:d(x,y))
```

6) Compléter le code de la fonction $\text{voisins}(E, x, d, k)$ définie ci-dessous.

```python
def voisins(E, x, d, k):
    """
    Entrée : ensemble d'apprentissage E (list), point x (tuple),
             distance d (function), nombre plus proches voisins k (int)
    Action : détermine les k plus proches voisins de x parmi les points de E
    Sortie : k plus proches voisins (list)
    """
    liste = tri(E, x, d) # liste triée (suivant distance d)
    ...                  # à compléter
```

Solution

```python
def voisins(E, x, d, k):
    """
    Entrée : ensemble d'apprentissage E (list), point x (tuple),
             distance d (function), nombre plus proches voisins k (int)
    Action : détermine les k plus proches voisins de x parmi les points de E
    Sortie : k plus proches voisins (list)
    """
    liste = tri(E, x, d) # liste triée (suivant distance d)
    return liste[:k]     # k points les plus proches de x
```

7) On détermine ensuite la classe dominante de $\text{voisins}(E, x, d, k)$, c'est-à-dire la classe i telle que l'intersection $E_j \cap \text{voisins}(E, x, d, k)$ contienne le maximum d'éléments pour $j = i$. Commenter chaque ligne du code suivant.

```python
def classe(E, x, d, k, p):
    """
    Entrée : ensemble d'apprentissage E (list), point x (tuple),
             distance d (function), nombre plus proches voisins k (int),
             classification p (dict)
    Action : détermine la classe majoritaire de x parmi les k plus proches voisins
    Sortie : classe majoritaire (int)
    """
    v = voisins(E, x, d, k)             # k plus proches voisins de x
    c = {p[y]:0 for y in v}             # liste des classes des voisins
    for y in v:                         # boucle sur voisins
        c[p[y]] += 1                    # comptage des classes
    maxi = 0                            # initialisation maximum
    for i in c:                         # boucle sur classes des voisins
        if c[i] > maxi:                 # si classe i plus nombreuse
            maxi, classmaj = c[i], i    # mise à jour classe majoritaire
    return classmaj                     # retourne classe majoritaire
```

Effectuer l'exécution pas à pas sur l'exemple simple `E=[(0.2,0.2),(0.8,0.8),(0.5,0.5)]`, `p={(0.1,0.1):'culture',(0.9,0.9):'économie',(0.5,0.5):'sport'}`, `k=1` et `x=(0.4,0.6)`.

8) Appliquer l'algorithme des k plus proches voisins à l'ensemble d'apprentissage `articles` et à la classification `sujets` et $x = (0.37, 0.69)$ pour $k = 1, 3, 7$. Dans quel sujet classer l'article x ?

Phase de test : matrice de confusion — On considère un ensemble test `articles_test` contenant $N = 20$ articles de presse à publier dont on connaît les sujets, répertoriés dans le dictionnaire `sujets_tests`. Voir lien ci-dessous.

https://cahier-de-prepa.fr/pc-theo/download?id=682

On souhaite évaluer la qualité de l'algorithme des k plus proches voisins en calculant la matrice de confusion M associée à cet ensemble test.

9) Rappeler la définition de la matrice de confusion M associée à un ensemble d'apprentissage E de classification p et à un ensemble test T.

Solution

> *On rappelle que la matrice de confusion est une matrice $k \times k$ dont les lignes représentent les classes réelles et les colonnes les classes prédites. L'élément M_{ij} de la matrice de confusion est le nombre d'articles de la classe i classés dans la classe j.*

10) On rappelle l'implémentation `python` du calcul de la matrice de confusion,

```python
def confusion(E, T, d, k, p, n):
    """
    Entrées : ensemble d'apprentissage E, ensemble de test T, distance d,
              nombre de voisins k, dictionnaire de classes p, nombre de classes n
    Sortie : matrice de confusion M
    """
    M = [[0]*n for l in range(n)]   # initialisation matrice
    for x in T:                     # boucle sur ensemble test
        i = p[x]                    # classe réelle
        j = classemaj(E,x,d,k,p)    # classe prédite
        M[i][j] += 1                # ajout au compte
    return M                        # retourne matrice de confusion
```

Appliquer cette fonction à l'ensemble test `articles_test` puis inspecter la matrice de confusion.

11) Déterminer la précision P et le taux d'erreur TE.